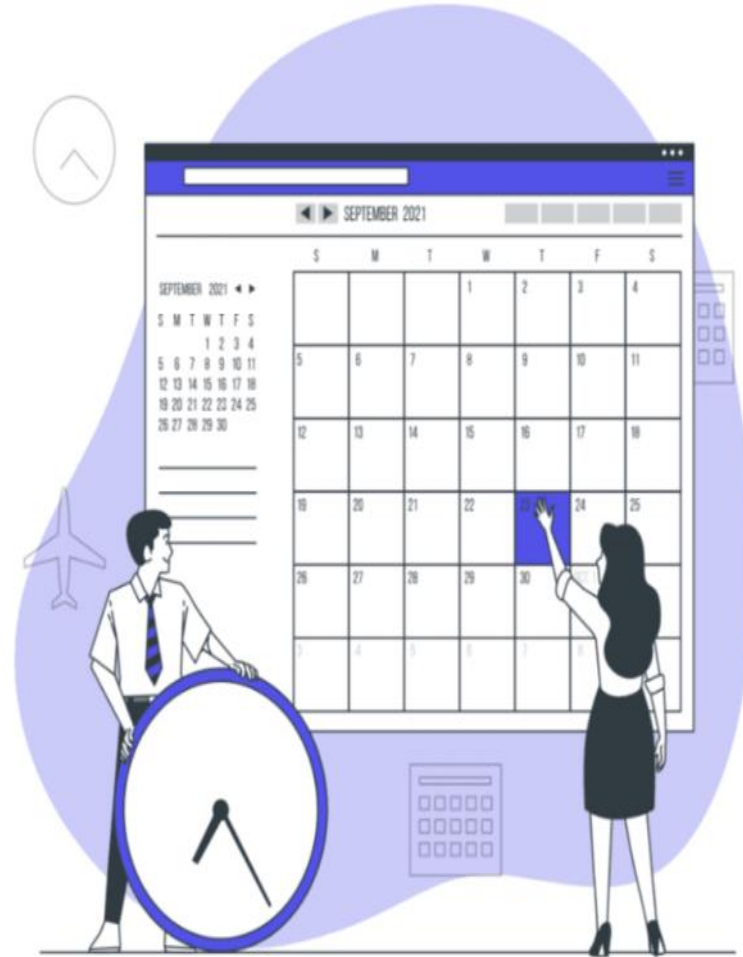


CHRONOS

A CALENDAR APPLICATION

Team 6 CS 3354.002

Abhay Srivatsa, Andrew Cross, Deeksha Patnam, Jonfernando Alfaro, Kayla Vu, Radhika Sharma, Sourish Pasula





PROJECT OBJECTIVE

An integrated mobile calendar solution for enterprise-wide event management and scheduling.

- **Key Features:**
 - Event creation/editing with alerts and categories
 - Multiple calendar views (Day, Week, Month, Agenda)
 - Private and shared calendar functionality
 - Conflict detection and time validation
 - Weekly recurring event support
- **Deliverable:** Enterprise-grade mobile calendar application integrated into company system using MVC architecture
- **Outcome:** Streamlined event management for 1,500-2,000 employees with secure, scalable solution and enhanced productivity

PROJECT SCOPE ASSUMP- TIONS

- Team of **7 software developers**
- Team is contacted by a mid-sized company to create a mobile calendar application to integrate into their system
- Mid-sized company:
 - ~1,500 to 2,000 Employees
 - ~38 IT Workers
 - ~\$38.5 million to \$1 billion annual company revenue
- Application will be developed for both iOS and Android devices

HARDWARE COST ESTIMATION

Category	Cost
Development Workstations	\$24,493
Testing Devices	\$12,000
Peripherals & Accessories	\$6,500
Infrastructure	\$5,000
Additional Hardware	\$1,200
Subtotal	\$49,193
Contingency (10%)	\$4,919
TOTAL ESTIMATED COST	\$54,112

- **Developer Workstations:** x7 M4 MacBook Pros 16" to serve as workstations for the 7 developers to develop the application
- **Testing Devices:** a variety of different phones, including iPhones latest and older models, and Androids from a variety of manufacturers at different price points for testing the Chronos app(8 iPhones + 11 Androids), as well as iPads and Android Tablets.
- **Peripherals:** x2 4k monitors per developer(14 total), wireless ergonomic keyboards, and productivity mice, laptop stands and docking stations.
- **Infrastructure:** Development server and NAS storage
- **Additional Hardware:** Charging stations, cables, and backups.
- **Justification:** For mobile application development targeting both iOS and Android platforms, developers require capable machines that can run development environments, emulators, and testing tools. The large variety of testing devices is necessary to test realistic end user use cases for a variety of screen sizes and manufacturers.

SOFTWARE COST ESTIMATION

- Software products that can be used such as
 - **Cloud Hosting:** estimated cost was around \$150/ month and since it a 5-6 month project; cost is around \$900
 - **Open-Source Software:** GitHub, Visual Studio Code (Development Tool), MySQL (Database) : total \$0
 - **Platform Development:**
 - Apple Development Program: \$99/year - used for iOS development (if we want to provide a mobile app on iOS)
 - o Apple Enterprise Developer: \$299/year
 - Google Play Developer Account: \$25 one time payment – used for Android app

Software Development Team



COST OF PERSONNEL

- **Estimated Personnel Cost**
- **Team:** 7 members (FE x2, BE x2, UI/UX, QA, DevOps)
- **Development Cost:** \$305k–540k (Avg ~\$350k, 6–9 months)
- **Post-Launch:** User guide + support = \$9k
- **Total Estimated Cost: \$359,000**
- **Assumptions:** US rates, 40-hr/week

Category	Costs (\$)
Development Team	350,000
Post-launch requirements	9000
Total	359,000

PROJECT COST ESTIMATE



Google Calendar as a microservices-based system—con-based system

Factors affecting cost:

- Database, server and cloud hosting
- Software complexity
- Microservices
- Developer exp.

FUNCTION POINT ESTIMATION	POINTS OF ESTIMATION	
	External Input (UI) External Output (UO) External queries Internal Logic Files External Interface Files	
	FOOTER	8

PROJECT TIMELINE

Phase 1: Requirements Definition

Document requirements and get feedback and opinions from employees and the company

~2 Weeks

Phase 2: Architecture Design

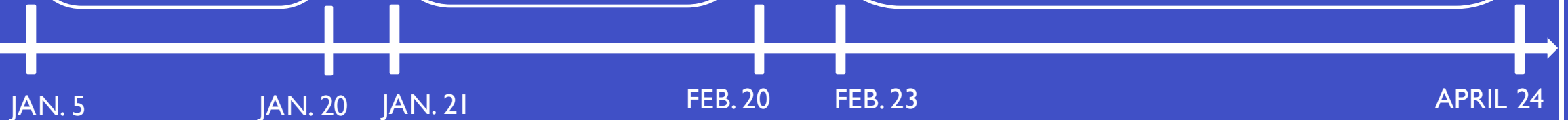
Plan roadmap, develop mock designs, defining milestones and developer roles

~4 Weeks

Phase 3: Developmental Phase

- Deploying incremental process:
 - Adding features one at a time
- Deploying an iterative process:
 - Test out features implemented to ensure no issues and gain feedback from users

~8-10 Weeks



PROJECT TIMELINE

Phase 4: Verification & Validation Testing

Keep testing features for the overall now completed application for more security issues and user feedback

~4 Weeks

Phase 5: Deployment

Prepare for official launch (finalize configurations, notify employees etc.) and train IT workers and other personnel for future support and maintenance

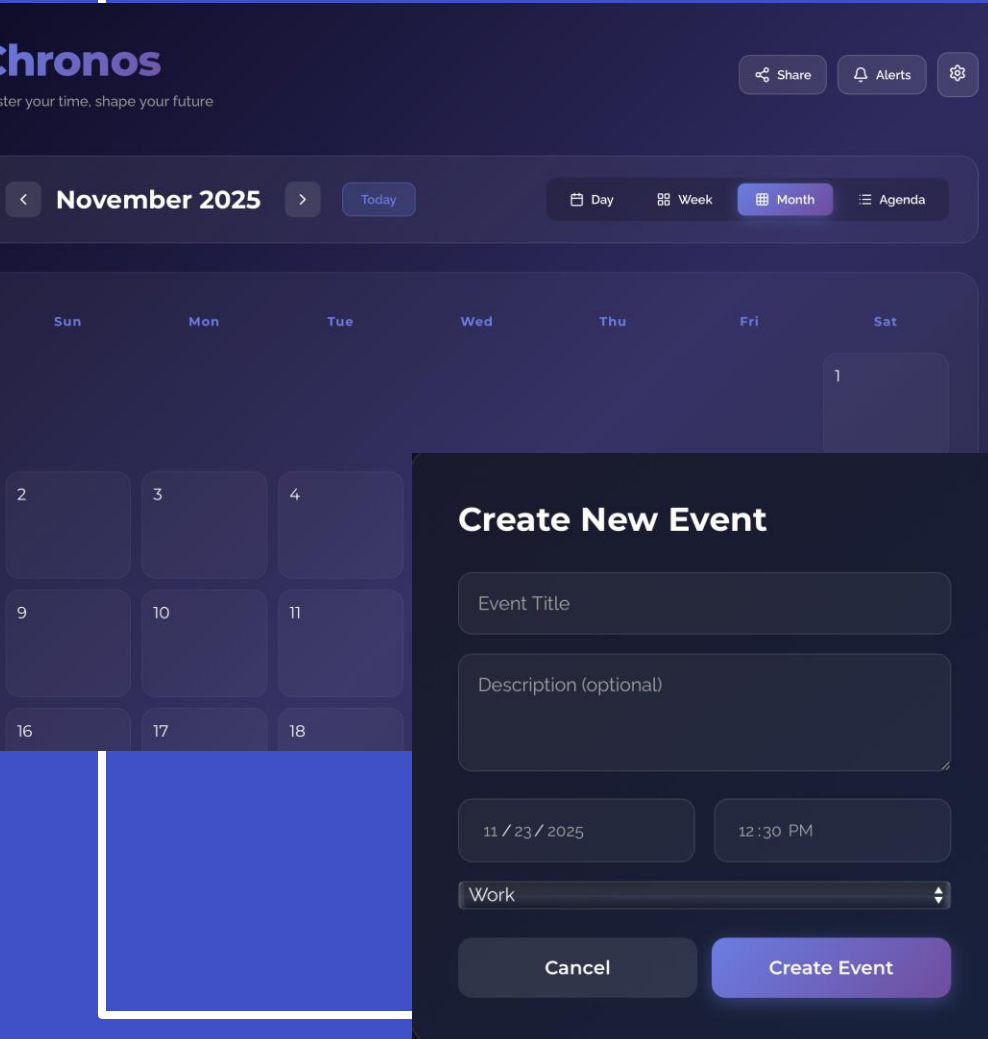
~2 Weeks

NOTES:

- Weekends and national holidays are NOT included in the schedule
- The working schedule for the project will be
 - 8 hours/day x 5 days a week = 40 hours a week
- At worst-case scenario, development of the application includes 1-2 months in its project timeline due to issues such as added requirements, verification/validation test issues etc.



FUNCTIONAL REQUIRMENTS



- Multiple calendar views (monthly, weekly, daily) with seamless switching
- Support for creating events with: title, description, start time, end time, date, optional location
- Automatic detection and alerts for overlapping or conflicting events
- Support for creating **recurring events** with customizable day patterns and date ranges
- Users can **edit or delete** events with confirmation prompts
- Users can create, edit, and delete **custom event categories**
- Events are **color-coded by category** across all calendar views
- System supports adding, editing, and deleting tasks in an agenda-style view
- Calendar can be shared with other users with appropriate permission levels

NOTE – THESE ARE JUST MOCKUPS

NON-FUNCTIONAL REQUIRMENTS

Usability

- Intuitive UI, minimal training
- Keyboard shortcuts
- Clear errors/confirmations

Performance

- Views load $\leq 2s$
- Event actions $\leq 1s$
- Supports **1000+** events

Storage

- $\leq 10 \text{ MB}$ per user
- Offline cache $\leq 50 \text{ MB}$

NON-FUNCTIONAL REQUIRMENTS

Reliability & Security

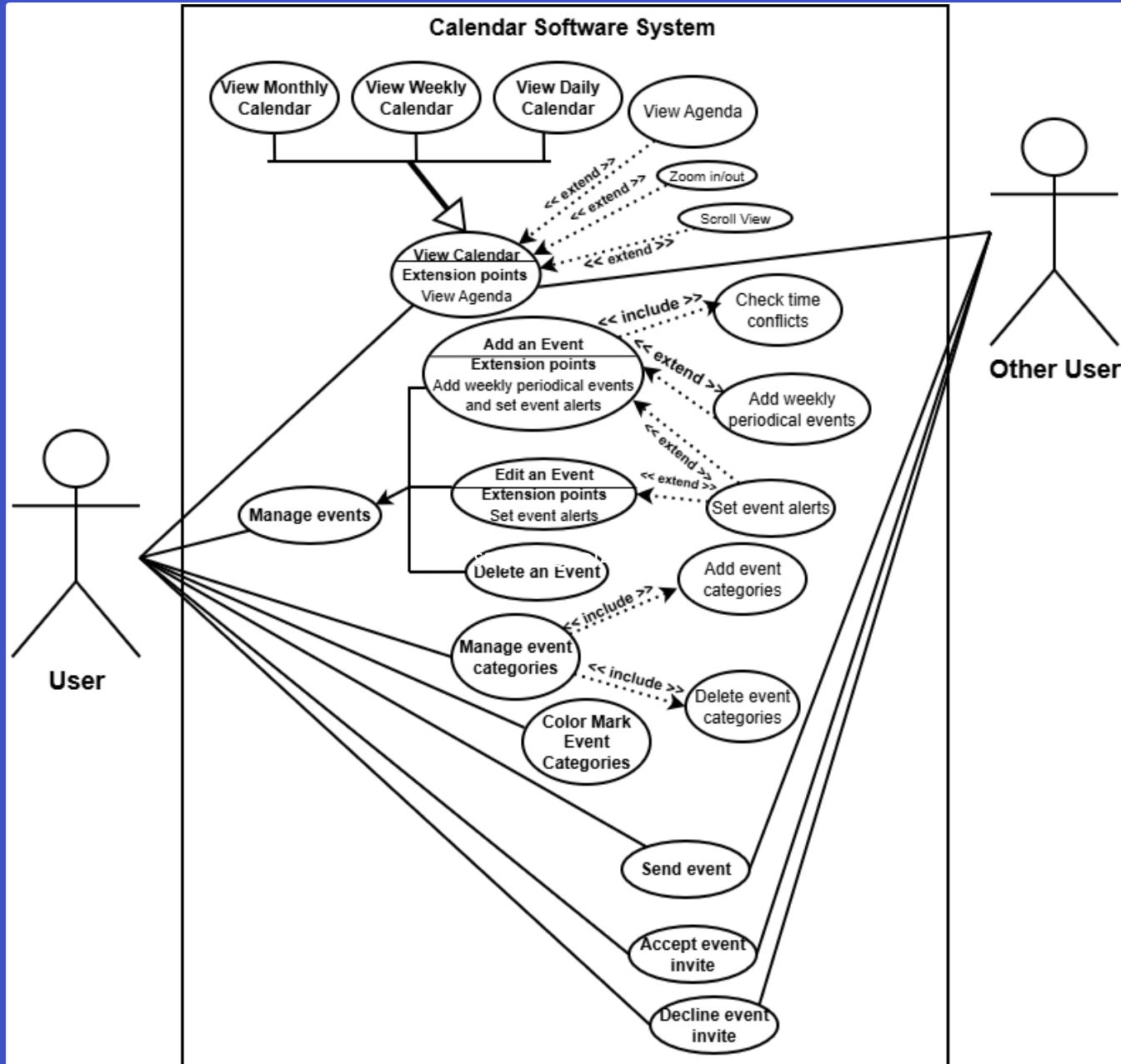
- Recovery < **6 hrs**; data restore < **24 hrs**
- Encrypted data; input validation
- Autosave every **30s**

Operational

- Latest 2 browsers; responsive $\geq$ **320×568**
- Works at **1 Mbps**
- **99% uptime**; daily backups

Development & Compliance

- **85%+** unit tests; automated integration tests
- CI + code reviews
- WCAG, privacy laws (HIPAA/COPPA/CCPA), OAuth
- Audit logs kept **180 days**



USE CASE DIAGRAM

1. Main User Actions:

- View Calendar in different modes(monthly, weekly, daily, agenda)
- Manage events (add, edit or delete)
- Manage event categories (add, delete, color-mark)

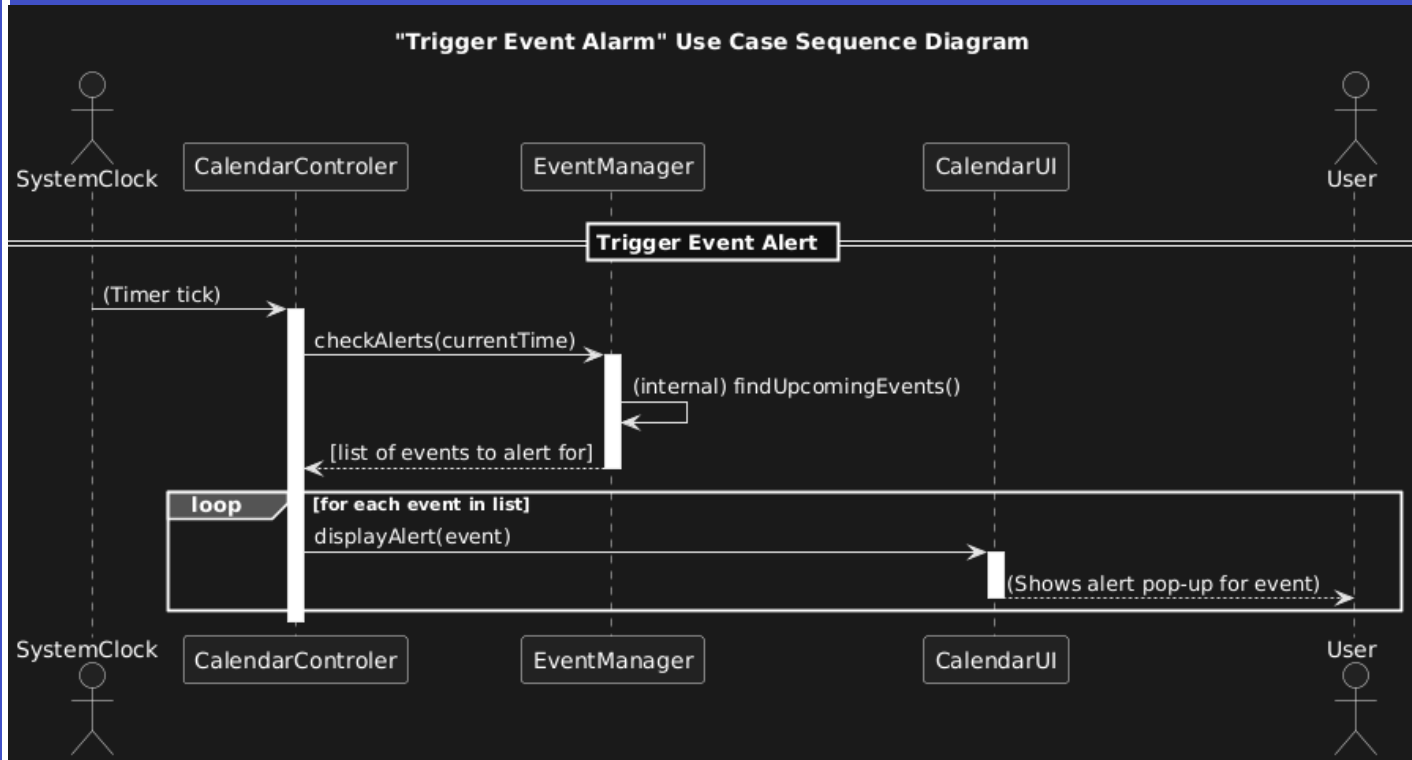
2. Collaboration Features:

- Send invite to other users
- Accept/decline events from other users

3. Actors:

- User (you) – performs all calendar and event functions
- Other users – receives and responds to invitations

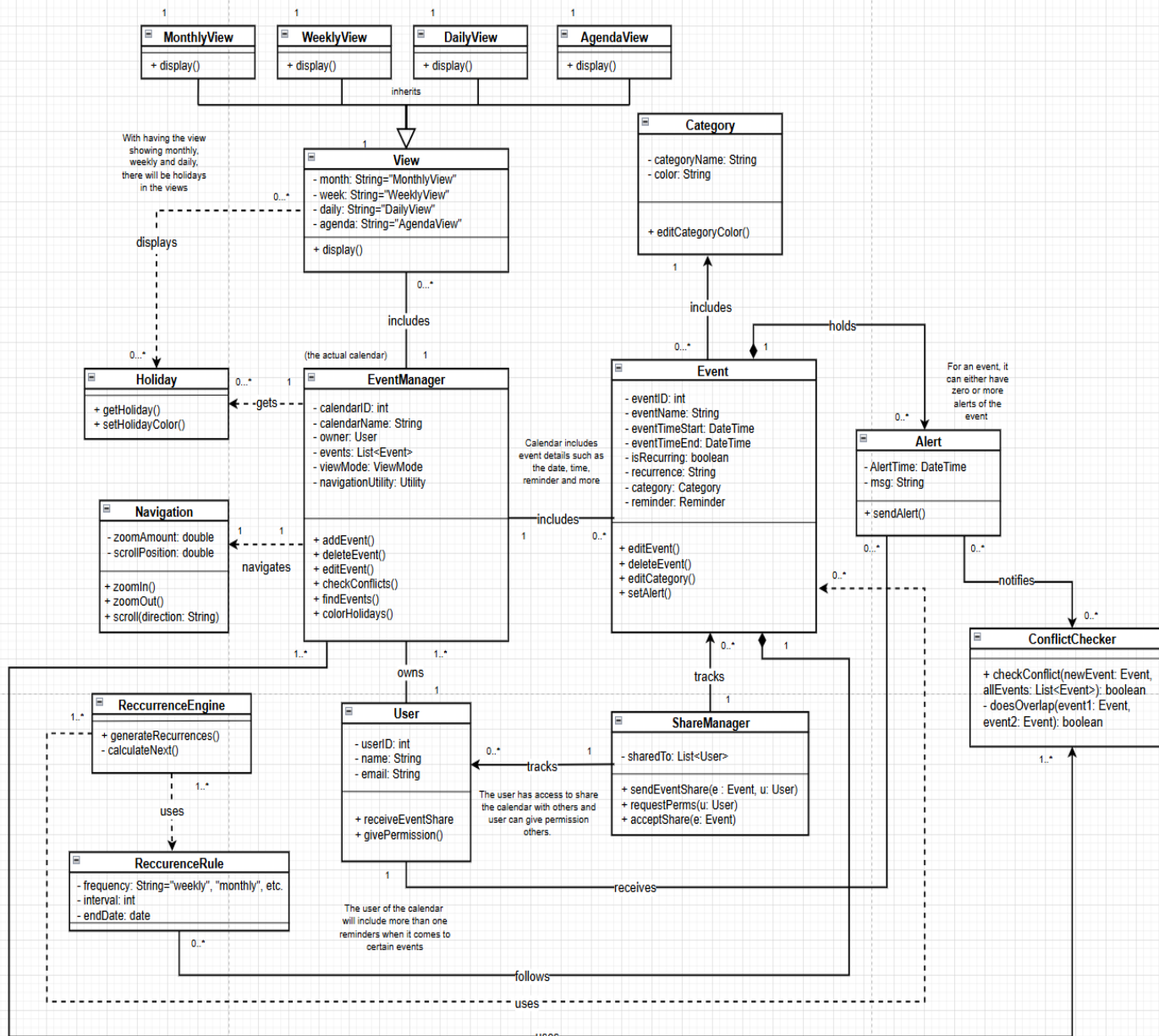
SEQUENCE DIAGRAM



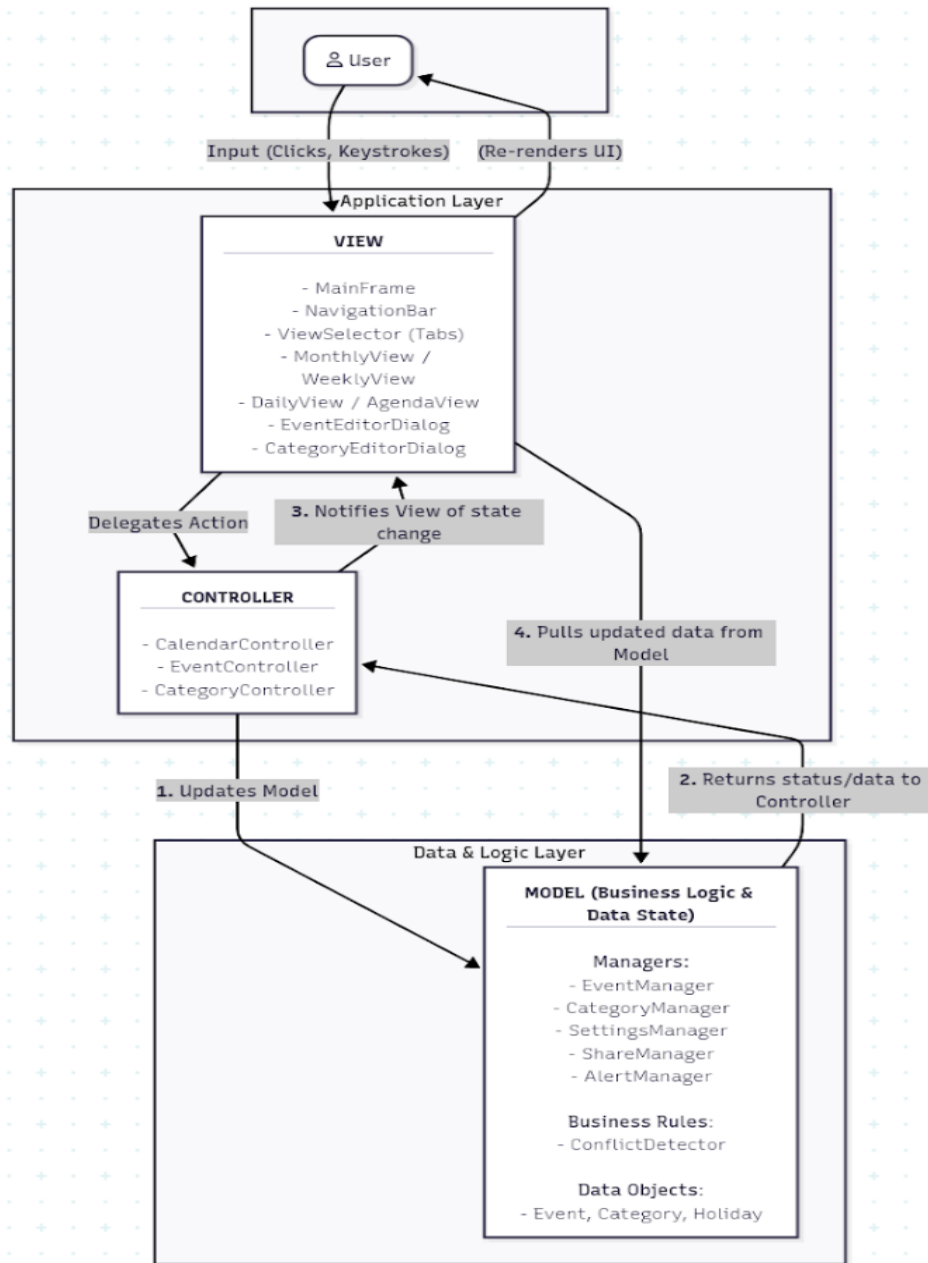
- Actors: Users, System Clock
- Components:
 - Controller: Calendar Controller
 - Model: Event Manager
 - View: CalendarUI
- Flow: System Clock → Controller: checks events → Model: retrieves data → Controller: processes → View: displays alert
- Outcome: Users receive timely notifications for upcoming events

Credit: PlantUML software used to create diagram

CLASS DIAGRAM



- Class diagram is a useful diagram due to showing a clear visual of the system structure
- Some of the relationships are shown on the diagram such as the month, weekly, daily and agenda view all inherit from the View class
- The features are included in each of the classes such as the ShareManager class allows the user to share events with others



ARCHITECTURAL DESIGN

Why MVC?

- Separates UI, logic, and data
- Makes the system scalable and easy to maintain
- **User - View:** User interacts with UI
- **View - Controller:** Actions are sent to controller
- **Controller - Model:** Business logic & data updated
- **Model - Controller:** Returns updated information
- **Controller - View:** UI notified to refresh
- **View:** Renders updated state



QUESTIONS?